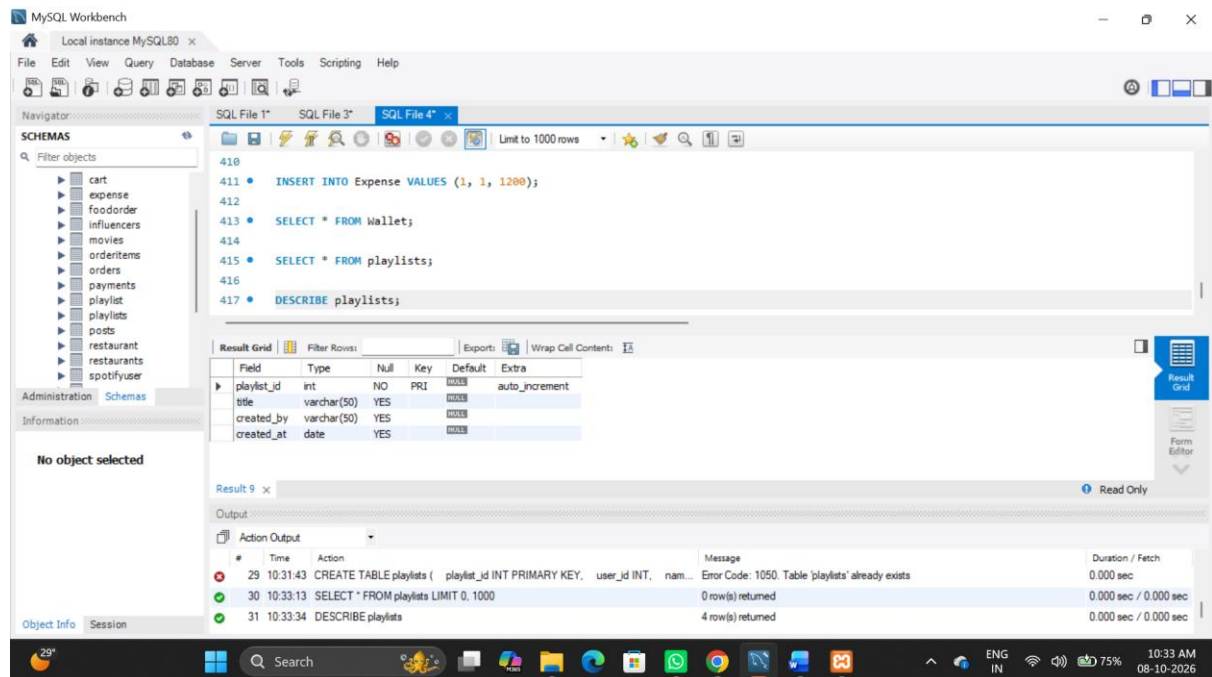


Mini project

Q1. Create a MySQL table called playlists to store Spotify-style music playlists, with columns: playlist_id (primary key), user_id (foreign key), name (varchar), and created_at (datetime). Write the CREATE TABLE statement.



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
410
411 • INSERT INTO Expense VALUES (1, 1, 1200);
412
413 • SELECT * FROM Wallet;
414
415 • SELECT * FROM playlists;
416
417 • DESCRIBE playlists;
```

The Result Grid shows the structure of the 'playlists' table:

Field	Type	Null	Key	Default	Extra
playlist_id	int	NO	PRI		auto_increment
title	varchar(50)	YES			
created_by	varchar(50)	YES			
created_at	date	YES			

The Output pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
29	10:31:43	CREATE TABLE playlists (playlist_id INT PRIMARY KEY, user_id INT, nam...	Error Code: 1050. Table 'playlists' already exists	0.000 sec
30	10:33:13	SELECT * FROM playlists LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
31	10:33:34	DESCRIBE playlists	4 row(s) returned	0.000 sec / 0.000 sec

Q2. Insert three sample records into the playlists table: one for a user with a 'Workout Mix', one with a 'Chill Vibes', and one with a 'Top Hits' playlist. Use realistic data for each field.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
FOREIGN KEY (user_id) REFERENCES Users(user_id));
429
430
431 • INSERT INTO playlists (playlist_id, user_id, name, created_at) VALUES
432 (1, 1, 'Workout Mix', '2026-10-08 08:30:00'),
433 (2, 2, 'Chill Vibes', '2026-10-08 10:15:00'),
434 (3, 3, 'Top Hits', '2026-10-08 12:00:00');
435
436 • SELECT * FROM playlists;
```

The Results Grid shows the output of the SELECT statement:

playlist_id	user_id	name	created_at
1	1	Workout Mix	2026-10-08 08:30:00
2	2	Chill Vibes	2026-10-08 10:15:00
3	3	Top Hits	2026-10-08 12:00:00
NULL	NULL	NULL	NULL

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
37	10:37:42	CREATE TABLE playlists (playlist_id INT PRIMARY KEY, user_id INT, nam...	0 row(s) affected	0.047 sec
38	10:38:08	INSERT INTO playlists (playlist_id, user_id, name, created_at) VALUES (1, 1, 'Work...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.016 sec
39	10:38:19	SELECT * FROM playlists LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

Q3. Write an SQL UPDATE statement to rename the 'Chill Vibes' playlist to 'Evening Chill' for a specific user_id.

The screenshot displays the MySQL Workbench interface. The SQL Editor contains the following code:

```
437
438 • UPDATE playlists
439   SET name = 'Evening Chill'
440   WHERE name = 'Chill Vibes'
441     AND user_id = 2;
442
443 • SELECT * FROM playlists
444   WHERE user_id = 2;
```

The Results window shows the output of the second query, displaying a table with the following data:

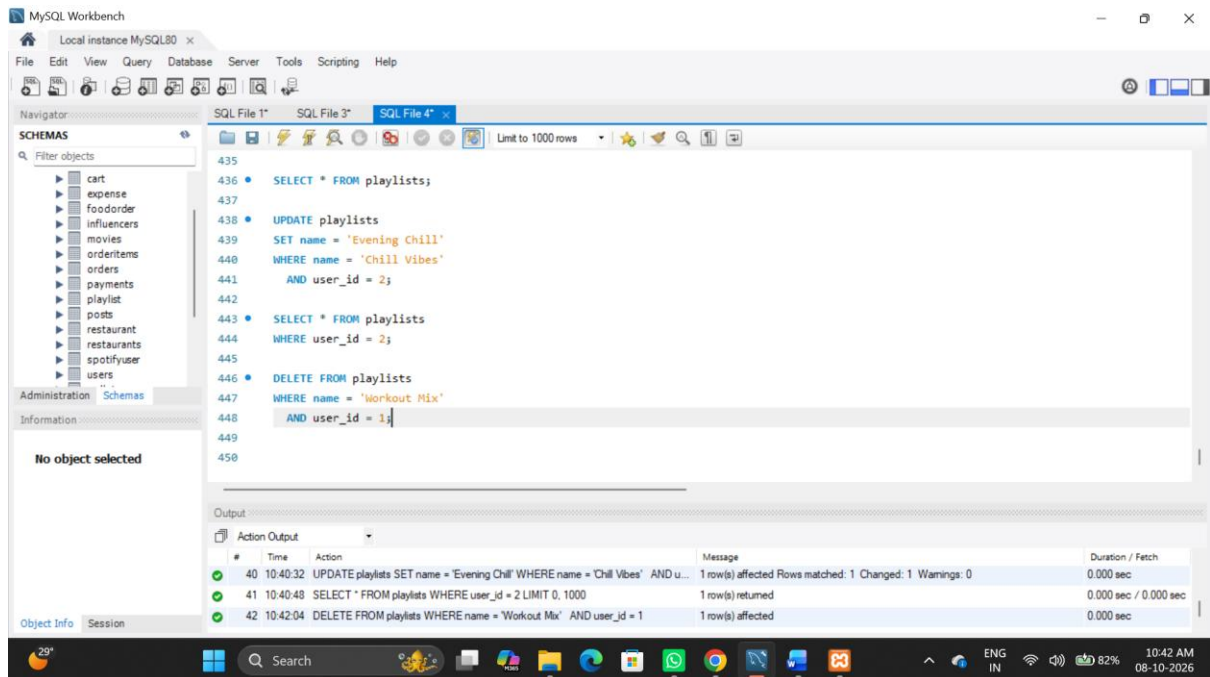
playlist_id	user_id	name	created_at
2	2	Evening Chill	2025-10-08 10:15:00

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
39	10:38:19	SELECT * FROM playlists LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
40	10:40:32	UPDATE playlists SET name = 'Evening Chill' WHERE name = 'Chill Vibes' AND u...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
41	10:40:48	SELECT * FROM playlists WHERE user_id = 2 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Q4. Write a DELETE statement to remove a playlist named 'Workout Mix' for a given user_id from the playlists table.

Hint: Make sure your WHERE clause targets only the intended row.



Q5. Create a stored procedure named GetMonthlyPlaylistCount that takes a user_id and a month (as an integer) as input, and returns the total number of playlists the user created in that month.

Hint: Use the MONTH() function in your WHERE clause.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following code:

```
457 FROM playlists
458 WHERE user_id = p_user_id
459 AND MONTH(created_at) = p_month;
460 END //
461
462 DELIMITER ;
463
464 CALL GetMonthlyPlaylistCount(2, 10);
```

The Results window shows a single row in the 'total_playlists' table:

total_playlists
1

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
42	10:42:04	DELETE FROM playlists WHERE name = 'Workout Mix' AND user_id = 1	1 row(s) affected	0.000 sec
43	10:44:19	CREATE PROCEDURE GetMonthlyPlaylistCount(IN p_user_id INT, IN p_mon...	0 row(s) affected	0.015 sec
44	10:44:43	CALL GetMonthlyPlaylistCount(2, 10)	1 row(s) returned	0.015 sec / 0.000 sec